Feedback and decisions: where this design came from

22 October 2026 · COMP1110 Group 08, Empty Classroom Plan · Square brackets [ ] are for the group to fill from their own notes

1. How this started

The problem did not come from a survey. It came up in ordinary conversation.

Members of the group had heard the same complaint from friends outside the team, more than once and from different people: between two classes there is nowhere obvious to sit, the library is full, and nobody knows which classroom is free right now. Some also mentioned small things about the rooms themselves, such as whether there are sockets, or whether the seating suits group work.

Those remarks are the reason we picked this problem. They are anecdotal: they tell us the problem exists among people we know, and they do not measure how common it is.

The design in the appendix then came from our own group discussion, starting from the Plan.

2. What friends mentioned, in passing

#	Who	What they said (in our words)	Where it shows up
1	[friend of which member]	[ ]	[ ]
2	[ ]	[ ]	[ ]
3	[ ]	[ ]	[ ]
4	[ ]	[ ]	[ ]

3. How strong this evidence is

What it is	What it is not
Comments made in conversation, recalled by group members	A survey, an interview study, or a measurement
Evidence that the problem exists in our own circle	Evidence of how common it is across HKU
A reason to build the thing	A result we can report

If the report needs a number, it has to come from the separate survey, not from this note. [confirm: which survey, and how many responses]

4. Decisions, and where they came from

The design was decided in group discussion. Write group in the last column, or friends, then group where a remark in section 2 prompted the row.

Decision	Why we made it	Source
The feed is a separate service with its own database, not a shared schema	Only a real boundary can be tested: network failure, slow responses, stale data	[ ]
The two systems share only the API contract	Either side can change without breaking the other; no coupling through code or files	[ ]
The service is read-only for us, with a separate write side for the registrar	We must not disturb the University's system, and we should not be able to	[ ]
Polling the metadata, not the timetable	The timetable changes a few times a day, not every second; a cheap GET /v1/meta tells us whether anything changed	[ ]
lastUpdated and revision exist as first-class fields	Without them the consumer cannot tell fresh data from stale, and cannot detect change cheaply	[ ]
Overlap rejection lives in the feed, not in our system	That rule belongs to the source of the timetable; enforcing it ourselves would pretend we own the data	[ ]
sockets is null when unknown, never false	We do not know and there are no sockets are different facts, and a student acts differently on each	[ ]

Decision	Why we made it	Source
Test switches for stale, slow and down	The behaviour we most need to test is failure behaviour, and it cannot be produced on demand otherwise	[ ]
A browser control panel served by the same service	We needed to see the timetable as a grid and change data quickly while developing	[ ]

5. What we discussed and deliberately did not do

• Pushing instead of polling. A university registry will not add a webhook for a student project. [confirm]

• Connecting to a University database directly. It would be intrusive and we have no permission. [confirm]

• Enforcing timetable rules in our own system. That would duplicate the source of truth. [confirm]

• A real login system for the feed. One fixed API key is enough to test with and without permission. [confirm]

6. What we know we cannot detect

A change in the meaning of a field cannot be caught automatically. If classes moved from ending at :50 to ending at :55, the data would still parse and our answer would be quietly wrong. Our answer is a diff against the previous revision with a threshold, plus a person looking at it. We wrote this down as a limitation instead of claiming the system handles it.

7. Who did what

Person	Contribution to this part
[name]	[ ]
[name]	[ ]
[name]	[ ]
[name]	[ ]
[name]	[ ]

Appendix: the architecture these decisions produce

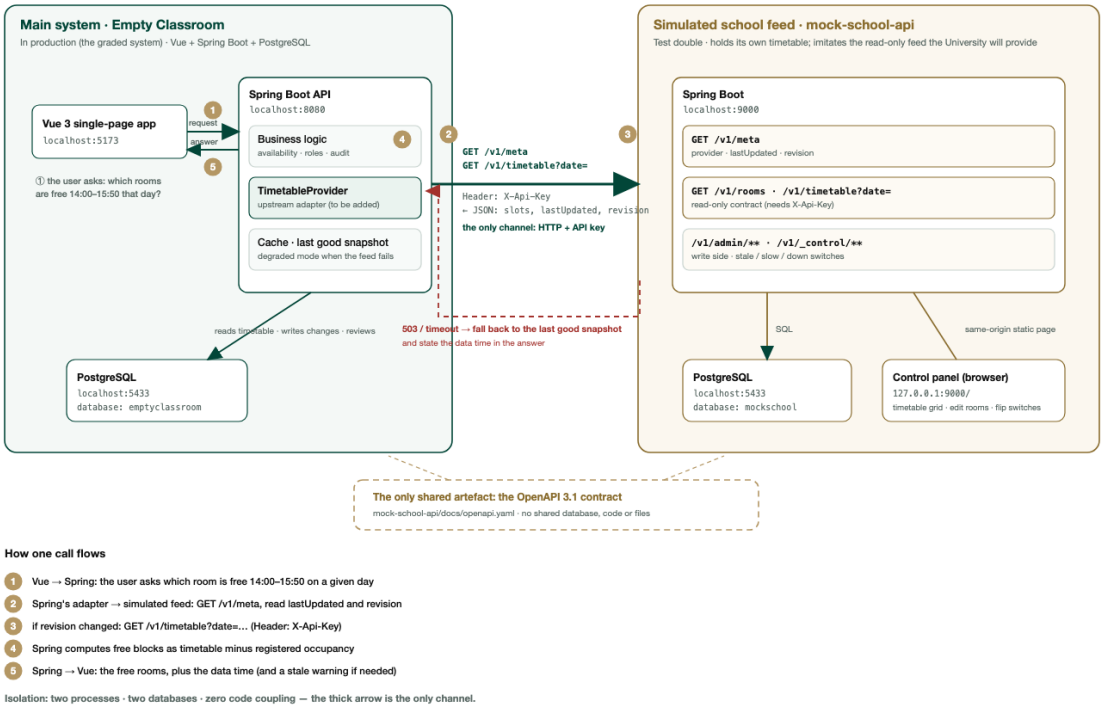


Figure 1. The main system and the simulated feed. Numbers 1 to 5 mark the call path.

The architecture is in figures/two-systems-architecture-en.png; the engineering reasoning is in design-log-2026-10-22.md.